Campus Reclaimed

Functional Testing Guide

Prepared for: Anna | Updated: October 2026
Version 2.0 — reflects Stage 3 tote pickup email workflow build

Overview

This guide covers functional testing across all five development stages of the Campus Reclaimed inventory and tote workflow system.

System Components:

- Frontend: React/Vite SPA → Docker → AWS ECR → AWS App Runner
- Backend: Supabase PostgreSQL + Deno/TypeScript Edge Functions (project ref: vkfzeojkxfuehgtwysrn)
- Email: Resend API (sent from info@campusreclaimed.com)
- Scheduling (Stage 5 only): Calendly integration

Six-Email Tote Workflow (Stage 3):

Email #	Trigger	Subject Line	Stage
Email 1	Student submits tote request	Campus Reclaimed – Reclaim Tote Request	Stage 3
Email 2	Ambassador marks tote delivered	Reclaim Tote Delivery	Stage 3
Email 3	Ambassador enters pickup window	Reclaim Tote Pick-Up	Stage 3
Email 4	Automated day-of reminder (7 AM ET daily)	Reminder: Reclaim Tote Pickup Today	Stage 3
Email 5	Tote picked up — donation student	Your Tote Was Received	Stage 3
Email 6	Tote picked up — consignment student	Your Tote Was Received	Stage 3

Stage Status Summary:

Stage	Description	Status	Test Sign-Off
Stage 1	Quick Wins & Payout Fixes	✓ **Deployed**	☐ Pass ☐ Fail
Stage 2	Database Migration & Shopify Channels	✓ **Deployed**	☐ Pass ☐ Fail
Stage 3	Tote Pickup Email Workflow (Emails 1–6)	Ready to Deploy	☐ Pass ☐ Fail
Stage 4	Ambassador Commissions & Monthly Recap	Not Yet Built	☐ Pass ☐ Fail
Stage 5	Calendly Integration (Scheduling)	Not Yet Built	☐ Pass ☐ Fail

Stage 1: Quick Wins & Payout Fixes

Status: ✓ Deployed and live

No re-deploy required for Stage 1 testing. These tests confirm existing deployed functionality.

Test	Description & Steps	Expected Result	Result
1A	**Email 1 — Student Tote Confirmation** When: Student submits the tote request form on Shopify. 1. Submit a tote request via the Shopify storefront form. 2. Check the inbox for the student email used.	**Subject: "Campus Reclaimed – Reclaim Tote Request"** • Opens with "Hi [First Name]," • Thanks student, says tote delivered in 24–48 hours • Includes link: campusreclaimed.com/pages/agreement • Signed "The Campus Reclaimed Team"	☐ Pass ☐ Fail
Notes: _____			
1B	**Ambassador Notification Email** When: Same request as Test 1A, check the assigned ambassador's inbox. 1. Check the assigned ambassador's email inbox. 2. Verify ambassador was assigned automatically (least-loaded active ambassador).	**Subject: "New Tote to Deliver — [Name] ([8-char ID])"** • Contains: student name, phone, delivery address • Instructs delivery within 24–48 hours • Assigned to least-loaded active ambassador	☐ Pass ☐ Fail
Notes: _____			
1C	**Storage Location Field** 1. Open the inventory app. 2. Find a consignment item. 3. Verify the Storage Location field is visible and editable. 4. Enter a location (e.g., "Bin A-3") and save.	• Storage Location field appears on the item detail view • Value saves and persists after page refresh	☐ Pass ☐ Fail
Notes: _____			
1D	**CSV Export** 1. Navigate to the export section in the inventory app. 2. Download the CSV. 3. Open in Excel or Numbers.	• CSV downloads without error • Columns include: item name, donor, price, status, storage location • All rows accounted for	☐ Pass ☐ Fail
Notes: _____			
1E	**Manual Payout Mark-as-Paid** 1. Find a donor with outstanding payout. 2. Click "Mark as Paid" in the payout section. 3. Refresh and verify.	• Payout status updates to "Paid" • Paid status persists after refresh • Payout record shows date paid	☐ Pass ☐ Fail
Notes: _____			

Stage 2: Database Migration & Shopify Channels

Status: ✓ Deployed and live

Test	Description & Steps	Expected Result	Result
2A	**Storage Location Persists to Database** 1. Set a storage location on an item (Test 1C). 2. Open Supabase Dashboard → Table Editor → items. 3. Confirm the storage_location column contains the value entered.	• storage_location column exists in items table • Value matches what was entered in app	☐ Pass ☐ Fail
Notes: _____			
2B	**Retail Price Field** 1. Open an item in the inventory app. 2. Enter a retail price. 3. Save and verify.	• Retail price field visible and editable • Saves correctly to database	☐ Pass ☐ Fail
Notes: _____			
2C	**Shopify — All 6 Sales Channels Published** 1. List a new item through the inventory app. 2. Open Shopify Admin and find the product. 3. Check "Sales channels and apps" panel.	• Item published to all 6 configured sales channels • No channels missing	☐ Pass ☐ Fail
Notes: _____			
2D	**Payout Override** 1. Find a donor record. 2. Enter a manual payout override amount. 3. Save and verify.	• Override amount saves correctly • Payout calculations reflect the override	☐ Pass ☐ Fail
Notes: _____			

Stage 3: Tote Pickup Email Workflow

Status: Code complete — deploy required before testing

Pre-Test Deploy Steps

Complete these steps before running any Stage 3 tests:

Step 1 — Run the migration SQL in Supabase Dashboard → SQL Editor:

```
ALTER TABLE tote_requests ADD COLUMN IF NOT EXISTS pickup_window TEXT;
ALTER TABLE tote_requests ADD COLUMN IF NOT EXISTS pickup_scheduled_at TIMESTAMPTZ;
ALTER TABLE tote_requests ADD COLUMN IF NOT EXISTS pickup_reminder_sent BOOLEAN DEFAULT false;
```

Step 2 — Deploy edge functions (run from your project directory):

```
supabase functions deploy tote-request --no-verify-jwt --project-ref vkfzeojkxfuehgtwysrn
supabase functions deploy tote-pickup-reminder --no-verify-jwt --project-ref vkfzeojkxfuehgtwysrn
```

Step 3 — Deploy frontend:

```
rm -rf dist && ./aws-deploy.sh
```

Testing without changing the Shopify form:

The Shopify form has not yet been updated to call the new edge function. Use this curl command to simulate a new tote request directly:

```
curl -X POST https://vkfzeojkxfuehgtwysrn.supabase.co/functions/v1/tote-request \
  -H "Content-Type: application/json" \
  -d '{"name":"Test Student","email":"YOUR-EMAIL@example.com",
      "phone":"555-1234","address":"123 Test Dorm, Room 101",
      "item_types":"Clothing, Shoes"}'
```

Test	Description & Steps	Expected Result	Result
3A	**DB Migration — New Columns Exist** Confirm the three new columns were added to tote_requests. 1. Open Supabase Dashboard → Table Editor → tote_requests. 2. Check column list.	**All three columns present:** • pickup_window (text) • pickup_scheduled_at (timestamptz) • pickup_reminder_sent (boolean, default false)	☐ Pass ☐ Fail
Notes: _____			
3B	**Email 1 — New Request Confirmation** When: Tote request submitted (via curl or Shopify form). 1. Run the curl command above with your test email. 2. Check the student inbox. 3. Check info@campusreclaimed.com inbox for admin notification. 4. Check the assigned ambassador's inbox.	**Student email:** • Subject: "Campus Reclaimed – Reclaim Tote Request" • Includes campusreclaimed.com/pages/agreement link • Says tote delivered in 24–48 hours **Admin email:** • Subject: "New Tote Request — [Name] ([ID])" **Ambassador email:** • Subject: "New Tote to Deliver — [Name] ([ID])"	☐ Pass ☐ Fail
Notes: _____			

Test	Description & Steps	Expected Result	Result
3C	**Email 2 — Tote Delivered** When: Ambassador marks tote as delivered in Tasks tab. 1. Open the inventory app Tasks tab. 2. Find the pending tote request from Test 3B. 3. Mark as "Tote Delivered" (status → tote_out). 4. Check the student inbox.	• Subject: "Reclaim Tote Delivery" • Body confirms tote delivered to address • Includes agreement link • Tells student they'll receive a pickup window email • Status updates to tote_out in app	☐ Pass ☐ Fail
Notes: _____			
3D	**Email 3 — Pickup Window Scheduled** When: Ambassador enters a pickup window in Tasks tab. 1. On the tote_out request, find the pickup window text input. 2. Enter: "Thursday, May 7th between 9 a.m. and 12 p.m." 3. Click "📅 Send Pickup Schedule". 4. Check student inbox. 5. Confirm pickup_window and pickup_scheduled_at saved in DB.	• Subject: "Reclaim Tote Pick-Up" • Body: "place tote outside your door, pick up tomorrow, [window]" • Asks student to reply to confirm • App shows: window text + "Reschedule" link • "Mark Tote Picked Up" button visible • DB: pickup_window and pickup_scheduled_at populated	☐ Pass ☐ Fail
Notes: _____			

3E	**Email 4 — Day-of Pickup Reminder**
	Note: Runs automatically at 7 AM ET daily via Supabase scheduler. Test manually here.
	1. In Supabase SQL Editor, backfill test request to simulate it was scheduled yesterday:
	`UPDATE tote_requests SET pickup_scheduled_at = NOW() - INTERVAL '1 day', pickup_reminder_sent = false WHERE id = '[your-request-id]' AND status = 'tote_out';`
	1. Trigger the function manually:
	`curl -X POST https://vkfzeojkxfuehgtwysrn.supabase.co/functions/v1/tote-pickup-reminder \ -H "Content-Type: application/json" -d '{}'`
	4. Check student inbox.
	5. Check DB: pickup_reminder_sent should now be true.
	6. Run curl again (idempotency check).
		• Subject: "Reminder: Reclaim Tote Pickup Today"
		• Body includes pickup window text
		• Asks student to place tote outside
		• DB: pickup_reminder_sent = true after send
		Idempotency:
		• Second curl run: {"sent": 0} — no duplicate email sent
			☐ Pass ☐ Fail

Notes: _____			
3F	**Email 5 — Tote Received (Donation Student)** *Prerequisite: Test student has agreement_type = "donation" in participation_agreements.* 1. In Tasks tab, find a tote_out request for a donation student.	• Subject: "Your Tote Was Received" • Body: thanks for donation, items being processed • No mention of consignment or agreement link	☐ Pass ☐ Fail

Test	Description & Steps	Expected Result	Result
	2. Click "Mark Tote Picked Up" (status → tote_picked_up). 3. Check student inbox.	• Status updates to tote_picked_up in app	
Notes: _____			
3G	**Email 6 — Tote Received (Consignment Student)** *Prerequisite: Test student has agreement_type = "consignment" in participation_agreements.* 1. In Tasks tab, find a tote_out request for a consignment student. 2. Click "Mark Tote Picked Up". 3. Check student inbox.	• Subject: "Your Tote Was Received" • Body: items being reviewed for consignment • Includes campusreclaimed.com/pages/agreement link • Different wording from Email 5	☐ Pass ☐ Fail
Notes: _____			

Stage 4: Ambassador Commissions & Monthly Recap

Status: Not yet built — test after implementation

Stage 4 adds commission tracking per ambassador and a monthly payout recap email. Tests below describe the expected behaviour; fill in the exact steps after implementation.

Test	Description & Steps	Expected Result	Result
4A	**Commission Rate Configuration** Ability to set a commission rate per ambassador (e.g., 10% per item sold). 1. Open an ambassador profile in the admin view. 2. Set or update the commission rate. 3. Save and verify.	• Commission rate field visible on ambassador profile • Rate saves to DB • Rate used in payout calculations	☐ Pass ☐ Fail
Notes: _____			
4B	**Ambassador Commission View** Ambassador can see their own pending and paid commissions. 1. Log in as an ambassador (or use admin view). 2. Navigate to the commissions section. 3. Verify amounts and item-level breakdown.	• Pending commissions listed with item detail • Paid commissions shown separately • Total amounts accurate	☐ Pass ☐ Fail
Notes: _____			
4C	**Admin — Mark Commission as Paid** 1. Open admin view of an ambassador's commission. 2. Click "Mark as Paid" for a pending commission. 3. Verify status update.	• Commission status changes to Paid • Paid date recorded • Ambassador's view updates	☐ Pass ☐ Fail
Notes: _____			
4D	**Monthly Payout Recap Email (Manual Trigger)** A monthly recap email sent to all ambassadors summarizing their items sold and commissions earned. 1. Trigger the monthly-payout-recap edge function manually. 2. Check ambassador inboxes.	• Each active ambassador receives one recap email • Email includes items sold count, total commission • Correct amounts per ambassador • No duplicates on re-trigger (idempotency)	☐ Pass ☐ Fail
Notes: _____			
4E	**pg_cron / Supabase Scheduler Verification** 1. Open Supabase Dashboard → Database → Extensions → verify pg_cron enabled. 2. Check Supabase Dashboard → Edge Functions → Schedules. 3. Confirm both scheduled functions are configured.	**Two scheduled functions active:** • tote-pickup-reminder — daily at 7 AM ET (cron: 0 11 * * *) • monthly-payout-recap — 1st of each month (to be configured)	☐ Pass ☐ Fail
Notes: _____			

Stage 5: Calendly Integration

Status: Not yet built — test after implementation

Stage 5 adds Calendly-based scheduling for students in on-campus dorms. Ambassadors set up a Calendly link; students in qualifying buildings receive that link in Email 1. Students in off-campus or non-qualifying buildings continue on the manual pickup window flow (Stage 3).

Test	Description & Steps	Expected Result	Result
5A	**Inside-Dorm Detection Logic** When a tote request address matches a known on-campus dorm building, the system flags it for Calendly scheduling. 1. Submit a tote request with an address matching a known dorm (e.g., "Johnson Hall"). 2. Check the tote request record in Supabase. 3. Submit a second request with an off-campus address.	• On-campus request: inside_dorm = true (or equivalent flag) • Off-campus request: inside_dorm = false • Detection based on address keywords or building list	☐ Pass ☐ Fail
Notes: _____			
5B	**Email 1 — Calendly Link for Inside-Dorm Requests** Inside-dorm students receive a Calendly link in their Email 1 to self-schedule pickup. 1. Submit a tote request with an inside-dorm address. 2. Check student inbox.	• Email 1 includes Calendly scheduling link • Link routes to the assigned ambassador's Calendly • Subject line unchanged: "Campus Reclaimed – Reclaim Tote Request"	☐ Pass ☐ Fail
Notes: _____			
5C	**Outside Dorm — Manual Scheduling (No Calendly)** Off-campus or non-qualifying students follow the Stage 3 manual pickup window flow unchanged. 1. Submit a tote request with an off-campus address. 2. Check student inbox. 3. Confirm pickup scheduling proceeds via Tasks tab (Email 3 flow).	• Email 1 does NOT include a Calendly link • Ambassador enters pickup window manually in Tasks tab • Email 3 flow proceeds as in Test 3D	☐ Pass ☐ Fail
Notes: _____			
5D	**Ambassador Calendly Account Setup** Each ambassador connects their Calendly account so the correct link is used per assignment. 1. Open an ambassador profile in admin. 2. Enter or link the ambassador's Calendly URL. 3. Submit a tote request assigned to that ambassador. 4. Verify Email 1 contains that ambassador's Calendly link.	• Calendly URL field on ambassador profile • Email 1 uses the assigned ambassador's link • Different ambassadors → different Calendly links	☐ Pass ☐ Fail
Notes: _____			
5E	**End-to-End Scheduling Flow (Inside Dorm)** 1. Submit inside-dorm tote request. 2. Student receives Email 1 with Calendly link. 3. Student books a time via Calendly.	• Student books via Calendly successfully • Ambassador receives booking notification	☐ Pass ☐ Fail

Test	Description & Steps	Expected Result	Result
	4. Verify ambassador receives Calendly booking notification. 5. Check that tote request in app reflects scheduled pickup.	• Pickup time visible in inventory app • Email 4 day-of reminder still fires on pickup day	
Notes: _____			

Final Sign-Off Checklist

Complete after all stages pass. Sign off on each stage before moving to the next.

Stage	Description	Tested By	Date	Sign-Off
Stage 1	Quick Wins & Payout Fixes			☐
Stage 2	Database Migration & Shopify Channels			☐
Stage 3	Tote Pickup Email Workflow (Emails 1–6)			☐
Stage 4	Ambassador Commissions & Monthly Recap			☐
Stage 5	Calendly Integration			☐

Notes & Known Gaps

Record any edge cases, failures, or deferred issues here:

Test	Issue / Note	Severity	Resolved